

Отчёт по выполнению задания «Функция $EFF_k(\alpha)$ для проверки $LR(k)$ -грамматик»

30 мая 2026 г.

1 Введение

В рамках данного задания требовалось запрограммировать функцию $EFF_k^G(\alpha)$, которая применяется при построении управляющих таблиц для $LR(k)$ -анализаторов. Функция $EFF_k^G(\alpha)$ позволяет прогнозировать аванцепочки для состояний сдвиг-свёртка при восходящем синтаксическом анализе (снизу-вверх).

2 Теоретическая база и математическая модель

Согласно формальному определению (теория формальных языков, слайд 80), функция $EFF_k^G(\alpha)$ определяется так:

1. Если α начинается с терминального символа: $EFF_k^G(\alpha) = FIRST_k^G(\alpha)$.
2. Если α начинается с нетерминала, то в множество EFF включаются префиксы длины $\leq k$ тех терминальных цепочек, которые выводимы из α правосторонним выводом ($\alpha \Rightarrow_{rm}^* \beta \Rightarrow_{rm} wx$), при условии, что **последний шаг этого вывода не использует ε -правило для самого левого символа α** .

Математически это эквивалентно рекурсивному усечению ε -веток у крайних левых узлов деревьев разбора. Таким образом, для любого нетерминала A :

$$EFF_k(A) = \bigcup_{A \rightarrow \beta \in P} \text{Contrib}(A \rightarrow \beta) \quad (1)$$

где вклад(Contrib) каждого правила определяется следующим образом:

- Если $\beta = \varepsilon$, то вклад равен $\emptyset$ (так как данная ветвь заканчивается ε -шагом, что исключается по условию).
- Если $\beta = c\gamma$ (c — терминал), вклад равен $FIRST_k(c\gamma)$.
- Если $\beta = B\gamma$ (B — нетерминал), вклад равен $EFF_k(B) \oplus_k FIRST_k(\gamma)$.

Для произвольной цепочки $\alpha = X_1X_2 \dots X_n$:

$$EFF_k(\alpha) = \begin{cases} \{\varepsilon\}, & \text{если } \alpha = \varepsilon \\ FIRST_k(\alpha), & \text{если } X_1 \in V_T \\ EFF_k(X_1) \oplus_k FIRST_k(X_2 \dots X_n), & \text{если } X_1 \in V_N \end{cases} \quad (2)$$

Такая формулировка позволяет алгоритмически точно находить искомое множество, избегая зацикливаний и строго исключая ε -концы.

3 Архитектура программного обеспечения

Приложение разработано на языке **C++** с использованием библиотеки **Qt** для реализации оконного графического интерфейса.

Структура проекта:

- **GrammarCore.h** — ядро алгоритма. Содержит структуры для хранения грамматики и итеративного вычисления множеств $FIRST_k$ и EFF_k на основе метода неподвижной точки. Реализована кастомная алгебра $\oplus_k$ конкатенации слов и множеств с усечением длины до k символов.
- **MainWindow.h / MainWindow.cpp** — интерфейс программы (GUI). Главное окно разбито на две части: панель ввода исходных данных (правила грамматики, значение k , исследуемая цепочка α) и панель вывода результатов. Ввод правил поддерживает альтернативы ($\mid$) и оператор « eps » для пустых цепочек.
- **main.cpp** — точка входа в программу.

4 Пример использования

Приложение было протестировано на грамматике из Примера 3.8:

```
S -> A B
A -> B a | eps
B -> C b | C
C -> c | eps
```

При $k = 2$ результат для $EFF_2(S)$ вычисляется программой как $\{ca, cb\}$, что строго подтверждает правильность её работы и согласуется с табличными выводами из теоретических материалов.

5 Заключение

В ходе выполнения задачи была написана программа на **C++** и **Qt**, которая успешно парсит входные КС-грамматики, составляет множества $FIRST_k$ и последовательно генерирует точные прогнозы функции EFF_k для заданных цепей α . Алгоритм четко обходит проблему ε -правил при правостороннем выводе, соответствуя строгой математической семантике задачи.